CHAPTER 6

DATA TYPES AND OPERATORS

CHAPTER ESSENCE AT A GLANCE:
Master the typing hierarchy and algebraic evaluation mechanics of C++ spanning primitive scalar types, operator precedence, prefix and postfix increments, and implicit type promotions.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Memory Footprint of Primitives:** Fundamental data types define the bit-width, representation range, and legal operations allocated to variables in memory.

• **Prefix versus Postfix Increment Mechanics:** Prefix increments modify the operand before expression evaluation, whereas postfix increments supply the original value before incrementing.

• **Ternary Conditional Branching:** The conditional operator evaluates a boolean condition to select between two expressions compactly without requiring full if-else block syntax.

• **Type Promotion Hierarchy:** Binary operations between mixed data types automatically promote smaller operands to the wider operand type to prevent precision loss during computation.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{Prefix: } ++x \to x = x + 1, \text{ use } x \quad | \quad \text{Postfix: } x++ \to \text{use } x, x = x + 1$$

Prefix versus Postfix Increment Semantics

$$\text{Ternary Expression} = (\text{Condition}) \, ? \, \text{ExprTrue} : \text{ExprFalse}$$

Conditional Operator Compact Branching

$$\text{Promotion: } \text{bool} \to \text{char} \to \text{int} \to \text{float} \to \text{double}$$

Implicit Numeric Type Promotion Hierarchy

$$\text{sizeof(char)} = 1 \leq \text{sizeof(int)} = 4 \leq \text{sizeof(double)} = 8$$

Primitive Memory Allocation in Bytes

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Watch out for integer division truncation: in C++, 5 / 2 evaluates to integer 2, not 2.5. To obtain floating precision, at least one operand must be float (5.0 / 2).

• **Alert:** In expressions like 'y = x++;', y receives the original value of x, and x increments afterward. In 'y = ++x;', x increments first and y receives the incremented value.

• **Alert:** Do not confuse assignment operator '=' with equality comparison operator '==' in conditional tests.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. Distinguish between fundamental data types and user-defined data types in C++ with examples. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Fundamental (Primitive) Data Types:** Built-in data types provided directly by the C++ language compiler (e.g. int, char, float, double, void). **[1 Mark]**

• **User-Defined Data Types:** Custom data types constructed by the programmer combining fundamental types to model complex entities (e.g. struct, class, union, enum). **[1 Mark]**

2. Explain the difference between prefix increment (++x) and postfix increment (x++) operators. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Prefix (++x):** Increments the operand value first by 1, and then uses the new updated value in the enclosing expression ('Change then use'). **[1 Mark]**

• **Postfix (x++):** Uses the original existing value of the operand in the enclosing expression first, and increments the operand afterward ('Use then change'). **[1 Mark]**

3. Explain the Conditional Operator (Ternary Operator ? :) in C++ with general syntax and an example. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Concept:** A ternary operator that takes three operands and serves as a compact alternative to simple if-else selection statements. **[1 Mark]**

• **Syntax:** condition ? expression1 : expression2; If condition evaluates to true, expression1 is returned; otherwise expression2 is returned. **[1 Mark]**

• **Code Example:** int maxVal = (a > b) ? a : b; sets maxVal to a if a is greater than b, else sets to b. Boxed Result: Compact inline conditional selection. **[1 Mark]**

4. Differentiate between implicit type conversion (type promotion) and explicit type conversion (type casting) with examples. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Implicit Conversion (Type Promotion):** Performed automatically by the compiler in mixed expressions where lower data types are promoted to higher types to prevent data loss (e.g. int + float results in float). **[1.5 Marks]**

• **Explicit Conversion (Type Casting):** Forcibly performed by the programmer using cast operators (e.g. float avg = (float) $\frac{total}{float}$count;), converting total temporarily to float before division. **[1.5 Marks]**

5. Explain relational operators and logical operators in C++ and state their return values. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Relational Operators:** Compare two values and evaluate to boolean true (1) or false (0); includes ==, != , <, >, <= , >= . **[1 Mark]**

• **Logical Operators:** Combine two or more conditional expressions: Logical AND (&& returns true if all operands are true), Logical OR (|| returns true if at least one operand is true), Logical NOT (! inverts truth value). **[1.5 Marks]**

• **Return Value:** All evaluate strictly to integer 1 (true) or integer 0 (false). **[0.5 Mark]**

6. Predict the output of the following C++ code snippet: int a = 5, b = 10, c; c = ++a + b++; cout << 'a=' << a << ' b=' << b << ' c=' << c; [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Evaluating ++a):** Prefix increment on a modifies a from 5 to 6 immediately; value 6 is used in expression. **[1 Mark]**

• **Step 2 (Evaluating b++):** Postfix increment uses original value 10 of b in expression; b then increments from 10 to 11. **[1 Mark]**

• **Step 3 (Evaluating c & Output):** c = 6 + 10 = 16. Final output: a=6 b=11 c=16. Boxed Result: a=6, b=11, c=16. **[1 Mark]**

7. Evaluate the arithmetic expression: 15 % 4 + $\frac{7}{2}$ × 3 - 1 adhering strictly to C++ operator precedence rules. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Modulo & Division):** Modulo and division have equal high precedence, evaluated left-to-right. 15 % 4 evaluates to remainder 3. $\frac{7}{2}$ integer division evaluates to 3. **[1 Mark]**

• **Step 2 (Multiplication):** 3 * 3 evaluates to 9. The expression reduces to: 3 + 9 - 1. **[1 Mark]**

• **Step 3 (Addition & Subtraction):** Evaluated left-to-right: 3 + 9 = 12; 12 - 1 = 11. Boxed Result: Final evaluated integer result is 11. **[1 Mark]**